# Real-Time Intrusion Detection for IoT Networks

## NEURAL IDS — a deep-learning NIDS on CIC-IoT-2023

Rareș Marta

Faculty of Mathematics and Computer Science
Babeș-Bolyai University
Coordinator: Alexandra-Ioana Albu

2026

# Why this matters

- Billions of IoT devices — cameras, plugs, sensors — shipped with weak, rarely-patched security.
- Each is an entry point for attackers: floods, scans, botnets like Mirai.
- The defence: detect attacks live, per flow.

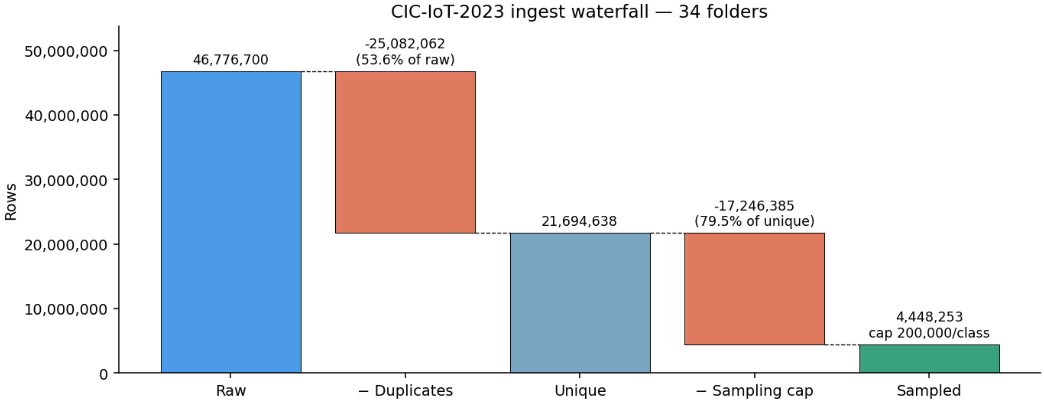
# What an IDS does

# The data: CIC-IoT-2023

**Canadian Institute for Cybersecurity**
**University of New Brunswick** (2023)

- 46.7M flows · 105 real IoT devices · 33 attacks in 7 families.
- 39 features — flow statistics, *no payload*.
- Count features leak the capture window — I treat them as leakage.

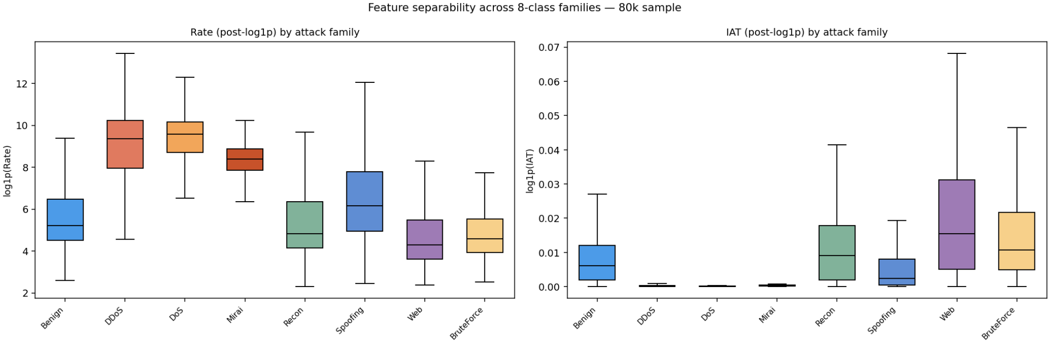
# Step 1 — clean 46.7M rows: dedup *before* sampling



# What we classify — 8 families

| | |
|---|---|
| **Benign** | the normal smart-home traffic we want to leave alone |
| **DDoS** | many devices flood a target at once |
| **DoS** | one machine floods a target until it falls over |
| **Mirai** | hacked IoT devices, taken over and remote-controlled |
| **Recon** | an attacker quietly mapping what's reachable |
| **Spoofing** | pretending to be someone else to redirect traffic |
| **Web** | attacks hidden inside web requests (SQL injection, XSS) |
| **BruteForce** | guessing passwords over and over until one works |

# Where the signal lives — floods separate, web overlaps benign

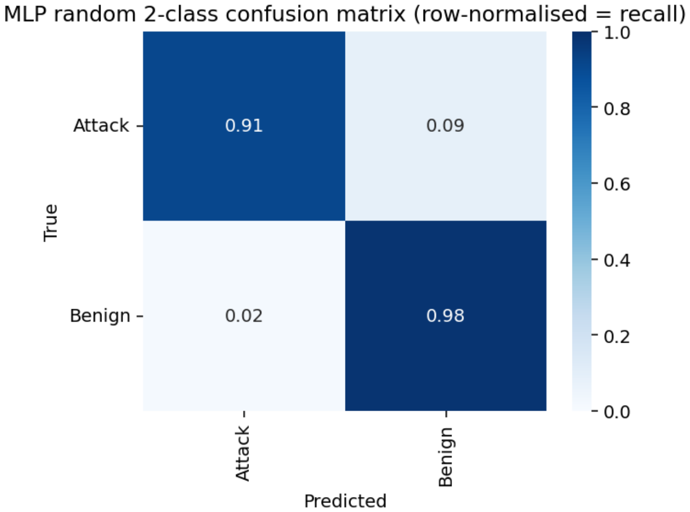


# Training pipeline — end to end

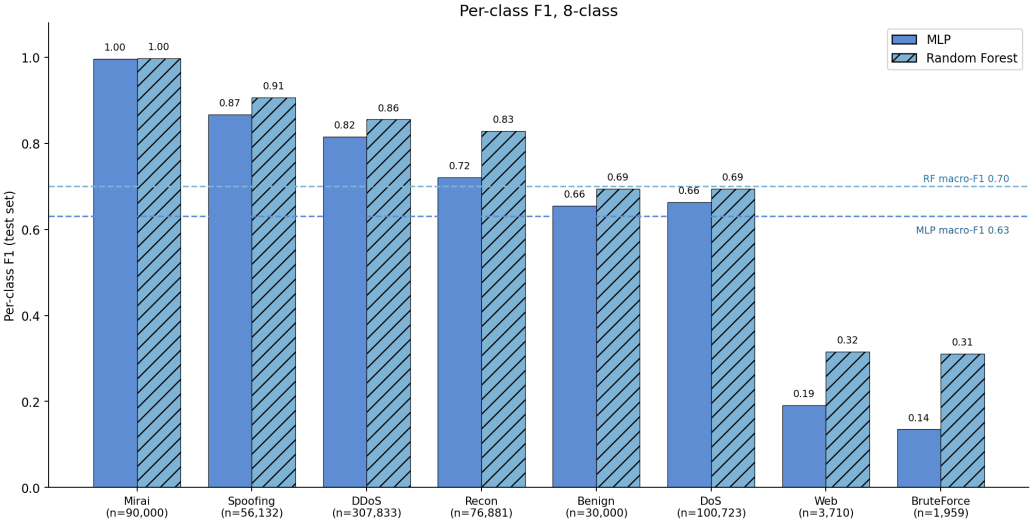
Ingest 46.7M → Dedup →21.6M → Prune 39→25 → RobustScaler + log1p → MLP (+ RF) → Evaluate 2- & 8-class

- Class-weighted loss for imbalance; tree baseline (Random Forest) on the same split.
- One stratified random split — comparable to the literature, but optimistic.

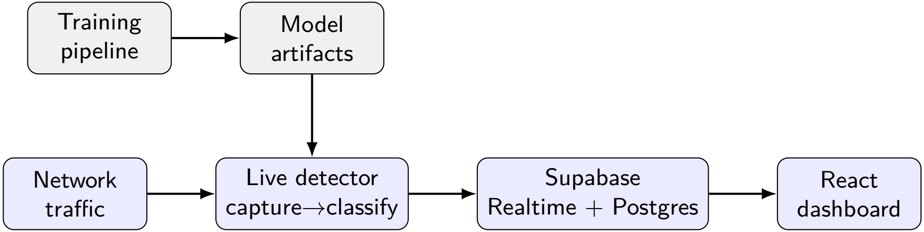
# Result — the binary gate (primary detect-&-alert task)



# Result — 8-class families: application-layer attacks collapse



# System architecture — as built




- Offline training emits artifacts; the live detector loads them and serves verdicts.
- Flows stream ephemerally over Realtime; incidents & stats persist in Postgres.

# Deployment choices — where each piece runs

| Piece | Runs on | Why |
|---|---|---|
| Detector + API | FastAPI on DigitalOcean droplet | serves both MLP & RF |
| Live sensor | same droplet (Docker) | captures real traffic |
| Backplane | Supabase (Postgres + Realtime) | push flows, no polling |
| Dashboard | React · Vite · Vercel | static, edge-served, CI deploy |

- Decoupled: training is offline; serving is a stateless API the UI calls over REST.

# Live demo

trigger an attack $\rightarrow$ watch it flag, per flow

# End-to-end — known-label captures

Run real captures whose correct labels we know in advance, end to end:

- Benign browsing $\rightarrow$ **98% Benign**
- SYN flood $\rightarrow$ **100% Attack**, mean confidence 99.6%, family = DoS
- Port scan $\rightarrow$ family = **Recon**

In-distribution the binary gate is reliable: **false-alarm rate 1.77%**.

# Conclusions & future work

## Delivered

- A defensible MLP baseline with tree comparison on CIC-IoT-2023.
- A working live, streaming IDS: capture → classify → dashboard.

## Limitations

- Out-of-distribution over-flagging (domain shift); dataset windowing leakage; transport-only features miss web attacks; optimistic random split.

## Future work

- Leakage-aware retraining; fine-tune on representative traffic; source allow-listing / baselining; line-rate deployment.